Ordered Set

The Only PBDS Tutorial You Need

Everything about ordered sets in GNU C++ PBDS — the standard way, every variation, every trick, every gotcha.

1 — What It Is

An ordered set is `std::set` with two extra O(log n) operations:

- `find_by_order(k)` — iterator to the k-th element (0-indexed).
- `order_of_key(x)` — count of elements strictly less than x.

Those two operations replace segment trees, BITs, and merge-sort trees for a huge class of problems.

---

2 — The Standard Setup

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<
    T,                                  // key type
    null_type,                          // null_type = set (not map)
    less<T>,                            // comparator
    rb_tree_tag,                        // ALWAYS red-black tree
    tree_order_statistics_node_update   // enables the two magic methods
>;
```

Works on every GCC-based judge — Codeforces, AtCoder, CodeChef, SPOJ. Does NOT compile on MSVC or Clang.

> bits/stdc++.h does NOT include the ext/ headers. You always need those two includes explicitly.

---

3 — The Two Core Operations

```
ordered_set<int> s;
s.insert(1); s.insert(5); s.insert(6); s.insert(17); s.insert(88);
// s = {1, 5, 6, 17, 88}
```

find_by_order(k) — get the k-th smallest element

```
*s.find_by_order(0)   // 1    (0th smallest)
*s.find_by_order(2)   // 6    (2nd smallest)
*s.find_by_order(4)   // 88   (4th smallest)
s.find_by_order(5)    // s.end() — OUT OF BOUNDS, do NOT dereference
```

order_of_key(x) — count of elements STRICTLY LESS than x

```
s.order_of_key(1)     // 0   (nothing < 1)
s.order_of_key(6)     // 2   (1, 5)
s.order_of_key(7)     // 3   (1, 5, 6)
s.order_of_key(25)    // 4   (1, 5, 6, 17)
s.order_of_key(100)   // 5   (all of them)
```

Derived Operations — build everything from these two

```
// Count elements <= x
s.order_of_key(x + 1)

// Count elements > x
s.size() - s.order_of_key(x + 1)

// Count elements >= x
s.size() - s.order_of_key(x)

// Count elements in [lo, hi]
s.order_of_key(hi + 1) - s.order_of_key(lo)

// Rank of x (1-indexed)
s.order_of_key(x) + 1

// k-th smallest (1-indexed)
*s.find_by_order(k - 1)

// k-th largest (1-indexed)
*s.find_by_order(s.size() - k)

// Minimum / Maximum
*s.find_by_order(0)
*s.find_by_order(s.size() - 1)

// Median
*s.find_by_order(s.size() / 2)           // upper median
*s.find_by_order((s.size() - 1) / 2)     // lower median

// Delete by index
s.erase(s.find_by_order(idx));
```

All standard `std::set` methods work — `insert`, `erase`, `find`, `lower_bound`, `upper_bound`, `size`, `begin`, `end`, range-for.

---

4 — Ordered Multiset (Allowing Duplicates)

The standard ordered set rejects duplicates. You will need duplicates very often. There are two approaches; one is correct and one is a trap.

4.1 — The Pair Trick (USE THIS)

Store `pair<value, unique_id>`. The pair's natural < keeps values sorted, and the unique id distinguishes duplicates.

```
ordered_set<pair<int, int>> ms;
int uid = 0;   // unique id counter

ms.insert({5, uid++});    // {5, 0}
ms.insert({2, uid++});    // {2, 1}
ms.insert({5, uid++});    // {5, 2}   <- duplicate value, different id
ms.insert({3, uid++});    // {3, 3}
ms.insert({2, uid++});    // {2, 4}
// Internal order: {2,1}, {2,4}, {3,3}, {5,0}, {5,2}
```

Querying — the key insight is what you pass:

```
// Count of elements with value < x
ms.order_of_key({x, 0})

// Count of elements with value <= x
ms.order_of_key({x + 1, 0})

// Count of elements with value == x
ms.order_of_key({x + 1, 0}) - ms.order_of_key({x, 0})

// Count in range [lo, hi]
ms.order_of_key({hi + 1, 0}) - ms.order_of_key({lo, 0})

// k-th smallest VALUE (0-indexed)
ms.find_by_order(k)->first       // .first strips the uid
```

Erasing one / all occurrences:

```
// Erase ONE occurrence of val:
auto it = ms.lower_bound({val, 0});
if (it != ms.end() && it->first == val)
    ms.erase(it);

// Erase ALL occurrences of val:
while (true) {
    auto it = ms.lower_bound({val, 0});
    if (it == ms.end() || it->first != val) break;
    ms.erase(it);
}
```

4.2 — The less_equal Approach (DANGEROUS)

Using less_equal<T> as comparator breaks find(), erase(value), and swaps the behavior of lower_bound/upper_bound. Only order_of_key and find_by_order work correctly. The pair trick is always safer. Use it.

Operation	What happens with less_equal
find(x)	Always returns end(). Never finds anything.
erase(x) by value	Undefined behavior. May crash.
lower_bound(x)	Acts like upper_bound (first element > x).
upper_bound(x)	Acts like lower_bound (first element >= x).
order_of_key(x)	Works correctly.
find_by_order(k)	Works correctly.

5 — Descending Order (Largest First)

```
using desc_set = tree<int, null_type, greater<int>,
                      rb_tree_tag, tree_order_statistics_node_update>;
```

Now `find_by_order(0)` returns the **largest** element, and `order_of_key(x)` counts elements **strictly greater** than x.

```
desc_set s;
s.insert(1); s.insert(5); s.insert(3); s.insert(7);
// Internal order: {7, 5, 3, 1}

*s.find_by_order(0)    // 7  (0th in descending = largest)
*s.find_by_order(3)    // 1  (3rd in descending = smallest)
s.order_of_key(5)      // 1  (one element > 5 under greater<>, i.e., 7)
```

> Most of the time you don't need this — you can use s.size() - s.order_of_key(x) on a normal ascending set to get the same information.

---

6 — Ordered Map (Key-Value with Order Statistics)

Change `null_type` to a value type:

```
template <typename K, typename V>
using ordered_map = tree<K, V, less<K>, rb_tree_tag,
                         tree_order_statistics_node_update>;

ordered_map<int, string> m;
m.insert({10, "ten"});
m.insert({3, "three"});
m.insert({7, "seven"});
// Sorted by key: {3:"three", 7:"seven", 10:"ten"}

auto it = m.find_by_order(1);
cout << it->first << " " << it->second;  // 7 seven

m.order_of_key(7);   // 1 (one key < 7)
```

Order statistics operate on **keys only**. Values are just along for the ride.

---

7 — Custom Struct as Key

You need `operator<` or a custom comparator. For `_GLIBCXX_DEBUG` mode, also define `operator<<`.

```
struct Point {
    int x, y;
    bool operator<(const Point& o) const {
        if (x != o.x) return x < o.x;
        return y < o.y;
    }
};

// For debug mode compatibility:
ostream& operator<<(ostream& o, const Point& p) {
    return o << "(" << p.x << "," << p.y << ")";
}

ordered_set<Point> pts;
pts.insert({1, 3});
pts.insert({2, 1});
pts.insert({1, 5});
// Sorted: {1,3}, {1,5}, {2,1}
pts.order_of_key({2, 0});  // 2  (two points < {2,0})
```

External comparator:

```
struct CmpByY {
    bool operator()(const Point& a, const Point& b) const {
        if (a.y != b.y) return a.y < b.y;
        return a.x < b.x;
    }
};

using y_sorted_set = tree<Point, null_type, CmpByY,
                          rb_tree_tag, tree_order_statistics_node_update>;
```

The comparator MUST define a strict weak ordering (a < b, never a <= b). If it doesn't, you get undefined behavior — silent corruption, not a compiler error.

---

8 — Strings, Vectors, Pairs

The ordered set is templated — it works with any type that has < defined.

```
// Strings
ordered_set<string> words;
words.insert("apple"); words.insert("banana"); words.insert("cherry");
words.order_of_key("banana");   // 1
*words.find_by_order(2);        // "cherry"

// Pairs (very common)
ordered_set<pair<int,int>> ps;
ps.insert({3, 1}); ps.insert({1, 4}); ps.insert({3, 2});
// Sorted: {1,4}, {3,1}, {3,2}
ps.order_of_key({3, 0});   // 1

// Vectors (yes, this works)
ordered_set<vector<int>> vs;
vs.insert({1, 2, 6}); vs.insert({1, 3, 3}); vs.insert({2, 6});
vs.order_of_key({1, 3});   // 1
```

---

9 — Split & Join

O(log n) on `rb_tree_tag`. Split a set around a value and merge two non-overlapping sets.

Split

```
ordered_set<int> a, b;
a.insert(1); a.insert(3); a.insert(5); a.insert(7); a.insert(9);
// a = {1,3,5,7,9}, b = {} (MUST be empty)

a.split(5, b);
// a = {1,3,5}     (elements <= 5 stay)
// b = {7,9}       (elements > 5 moved to b)
```

Join

```
ordered_set<int> a, b;
a.insert(1); a.insert(3); a.insert(5);
b.insert(7); b.insert(9);

a.join(b);
// a = {1,3,5,7,9}, b = {} (emptied)
// REQUIREMENT: max(a) < min(b) — ranges must NOT overlap
```

Extracting a range [lo, hi]

```
ordered_set<int> s, right_part, middle;
// s = {1, 3, 5, 7, 9, 11}

s.split(hi, right_part);       // right_part gets everything > hi
s.split(lo - 1, middle);       // middle gets [lo, hi]

// Put back:
s.join(middle);
s.join(right_part);
```

split destination MUST be empty (otherwise: UB). join requires non-overlapping key ranges (otherwise: exception). Only rb_tree_tag gives O(log n) — splay_tree_tag and ov_tree_tag take O(n).

---

10 — Practical Problem Patterns

10.1 — Count Inversions

Given array a[], count pairs (i, j) with i < j and a[i] > a[j]. Time: O(n log n).

```
long long count_inversions(vector<int>& a) {
    ordered_set<pair<int,int>> s;
    long long inv = 0;
    for (int i = 0; i < (int)a.size(); i++) {
        inv += s.size() - s.order_of_key({a[i], i});
        s.insert({a[i], i});
    }
    return inv;
}
```

10.2 — Sliding Window Median

```
void sliding_median(vector<int>& a, int k) {
    ordered_set<pair<int,int>> s;
    for (int i = 0; i < (int)a.size(); i++) {
        s.insert({a[i], i});
        if (i >= k)
            s.erase(s.lower_bound({a[i - k], i - k}));
        if (i >= k - 1)
            cout << s.find_by_order((k - 1) / 2)->first << " ";
    }
}
```

10.3 — Dynamic k-th Smallest (Online)

```
ordered_set<pair<int,int>> s;
int uid = 0;

void add(int x)    { s.insert({x, uid++}); }
void remove(int x) {
    auto it = s.lower_bound({x, 0});
    if (it != s.end() && it->first == x) s.erase(it);
}
int kth(int k)     { return s.find_by_order(k-1)->first; }  // 1-indexed
int rank_of(int x) { return s.order_of_key({x, 0}) + 1; }  // 1-indexed
int count_range(int lo, int hi) {
    return s.order_of_key({hi+1,0}) - s.order_of_key({lo,0});
}
```

10.4 — Count Greater After Each Insertion

```
ordered_set<pair<int,int>> s;
int uid = 0;
int n; cin >> n;
while (n--) {
    int x; cin >> x;
    cout << s.size() - s.order_of_key({x + 1, 0}) << "\n";
    s.insert({x, uid++});
}
```

10.5 — When Ordered Set Replaces a Segment Tree / BIT

If your problem only needs: insert/delete, query k-th smallest, query rank, and count elements in a range — then an ordered set replaces a Fenwick tree + coordinate compression. Less code, handles any value range, same O(log n).

> Trade-off: ordered sets use ~5x more memory than a BIT. For n <= 10^6 this is fine. For n > 5*10^6 you might hit memory limits.

10.6 — Counting Distinct Elements in a Range (Offline Sweep)

Problem: Given an array a[] and Q queries, each asking "how many distinct values are in a[l..r]?"

Key idea: For each distinct value, we only care about its **rightmost (latest) occurrence**. If value 5 appears at positions 1, 4, and 7, only position 7 matters — because any query range that includes position 7 already "sees" value 5. The earlier positions are redundant duplicates.

What the ordered set stores: It holds **positions** (array indices) — specifically, the position of the last occurrence of each value seen so far. At any point during the sweep, the set contains exactly one position per distinct value. So the set's size equals the count of distinct values seen.

The algorithm:

- Sort all queries by their right endpoint r.
- Sweep through the array left to right (r = 0, 1, 2, ...).
- At each position r: if a[r] was seen before at position p, erase p from the set (it's stale — position r is the new "latest" for this value). Then insert r.
- To answer query [l, r]: count how many positions in the set fall within [l, r]. Since we haven't inserted anything past r yet, we just need positions >= l. That's: s.size() - s.order_of_key(l).

Walkthrough:

```
Array: a[] = {3, 1, 3, 2, 1}
              0  1  2  3  4    (positions)

Query: [1, 4] -> distinct values in {1, 3, 2, 1}? Answer: 3

r=0: a[0]=3, new value.       Set = {0}         (pos 0 -> val 3)
r=1: a[1]=1, new value.       Set = {0, 1}      (pos 0->3, pos 1->1)
r=2: a[2]=3, seen at pos 0!   Erase 0, insert 2.
                              Set = {1, 2}      (pos 1->1, pos 2->3)
r=3: a[3]=2, new value.       Set = {1, 2, 3}   (pos 1->1, 2->3, 3->2)
r=4: a[4]=1, seen at pos 1!   Erase 1, insert 4.
                              Set = {2, 3, 4}   (pos 2->3, 3->2, 4->1)

Answer query [1, 4] at r=4:
  s.size() - s.order_of_key(1)  =  3 - 0  =  3
  Positions {2, 3, 4} are all >= 1.
  They represent values {3, 2, 1} -> 3 distinct values. Correct!
```

Why s.size() - s.order_of_key(l)? order_of_key(l) counts positions strictly less than l — those fall outside [l, r]. Nothing beyond r exists in the set (we haven't swept there yet). So total minus too-small = positions in [l, r] = distinct values in [l, r].

Full implementation:

```
struct Query { int l, r, idx; };

vector<int> solve(vector<int>& a, vector<Query>& queries) {
    int n = a.size(), q = queries.size();
    vector<int> ans(q);
    sort(queries.begin(), queries.end(),
         [](auto& a, auto& b){ return a.r < b.r; });

    ordered_set<int> s;
    map<int, int> last_pos;  // value -> last position seen
    int qi = 0;

    for (int r = 0; r < n && qi < q; r++) {
        if (last_pos.count(a[r]))
            s.erase(last_pos[a[r]]);  // remove stale position
        last_pos[a[r]] = r;
        s.insert(r);

        while (qi < q && queries[qi].r == r) {
            int l = queries[qi].l;
            ans[queries[qi].idx] = s.size() - s.order_of_key(l);
            qi++;
        }
    }
    return ans;
}
```

Each position is inserted and erased at most once, so total work is O((n + q) log n).

11 — Gotchas & Pitfalls

order_of_key is STRICTLY LESS, not <=

This is the #1 source of bugs. order_of_key(5) does NOT count 5 itself. To get count <= 5, use order_of_key(6).

find_by_order with out-of-bounds index

Returns end(). Dereferencing end() is UB. Always check: if (s.find_by_order(k) == s.end()) { /* k >= s.size() */ }

No duplicates in standard ordered set

s.insert(5); s.insert(5); — second insert does nothing. Size stays 1. Use the pair trick.

split destination must be empty

If you split into a non-empty set: undefined behavior — may crash, may corrupt.

join requires non-overlapping ranges

If any key in the argument lies between keys in 'this': throws exception or UB.

Memory usage

~40-80 bytes per element. For n = 10^6 that's ~40-80 MB. For n = 5*10^6 you might hit limits.

GCC-only

Does not compile on Clang or MSVC. On Mac, install GCC via Homebrew.

_GLIBCXX_DEBUG mode

Ordered set on custom structs fails to compile unless you define operator<< for your struct.

Integer overflow with order_of_key

order_of_key(x + 1) where x = INT_MAX causes overflow. Use long long as key type.

---

12 — Complexity Cheat Sheet

Operation	Time
insert	O(log n)
erase (by value or iterator)	O(log n)
find	O(log n)

lower_bound / upper_bound	O(log n)
find_by_order(k)	O(log n)
order_of_key(x)	O(log n)
split	O(log n)
join	O(log n)
size	O(1)
Iteration (range-for)	O(n)
Memory per element	~40-80 bytes

---

13 — Copy-Paste Template

```
#include <bits/stdc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace std;
using namespace __gnu_pbds;

// -- Ordered Set (unique elements) --
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
                         tree_order_statistics_node_update>;
// s.find_by_order(k)  -> iterator to k-th smallest (0-indexed)
// s.order_of_key(x)   -> count of elements strictly < x

// -- Ordered Multiset (via pair trick) --
// Use: ordered_set<pair<int, int>>
// Insert:            s.insert({val, uid++})
// Count < x:         s.order_of_key({x, 0})
// Count <= x:        s.order_of_key({x + 1, 0})
// Count in [lo,hi]:  s.order_of_key({hi+1,0}) - s.order_of_key({lo,0})
// k-th smallest:     s.find_by_order(k)->first
// Erase one of val:  auto it = s.lower_bound({val,0});
//                    if (it->first==val) s.erase(it);

// -- Ordered Map (key-value with order stats on keys) --
// template <typename K, typename V>
// using ordered_map = tree<K, V, less<K>, rb_tree_tag,
//                          tree_order_statistics_node_update>;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    return 0;
}
```

14 — When NOT to Use Ordered Set

- **Need sum/min/max of a range?** Ordered set only gives count-based queries. Use a segment tree or BIT.
- **Need persistent versions?** No persistent PBDS. Use a persistent segment tree.
- **Need to merge two overlapping sets?** join requires non-overlapping ranges. Insert one-by-one if ranges overlap.
- **n > 5 x 10^6?** Memory might be too tight. Fall back to a BIT with coordinate compression.
- **Non-GCC judge?** Won't compile. Write your own balanced BST or use a different approach.

For everything else — dynamic rank queries, k-th element, counting in ranges, inversions, sliding medians — the ordered set is the fastest thing you can code.